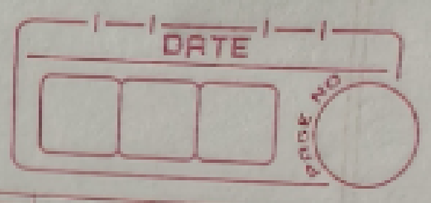
# Chap - 5.




Q1) Which factors are considered to evaluate best index technique?

→ Indexing technique is used to locate and access data in a table.
eg - An index in a book.
Instead of scanning every page, the database can use use the index to jump directly to the location of the required data.
- Words in the index are in sorted order, making it easy to find the word we are looking for.
- Index is much smaller than book, further reducing the effort needed to find the words we are looking for.
- There are two types of indices
1) Ordered indices:
These are based on sorted ordering of the values.
2) Hash indices:
These are based on an uniform distribution of values across a range of buckets.
- The bucket to which a value is assigned is determined by a function, called as Hash Function.

→ Each Technique must be evaluated on the basis of following factors.
1) Access types:
↳ It means the types of access that are supported efficiently.
↳ Access types can include finding records with a specified attribute value and finding

records whose attribute values fall in a specified range.

2) Access Time:

The time it takes to find a particular data item or set of items is Access Time.

3) Insertion Time:

The time required to insert a new data item is called Insertion time.

↳ This value includes the time it takes to find the correct place to insert the new data item and the time it takes to update the index structure.

4) Deletion Time:

The time required to delete a data item is called Deletion time.

↳ This value includes the time it takes to find the item to be deleted and the time it takes to update the index structure.

5) Space overhead:

The additional space occupied by an index structure is called Space overhead.

↳ Provided that the amount of additional space is moderate, it is usually worth-while to sacrifice the space to achieve improved performance.

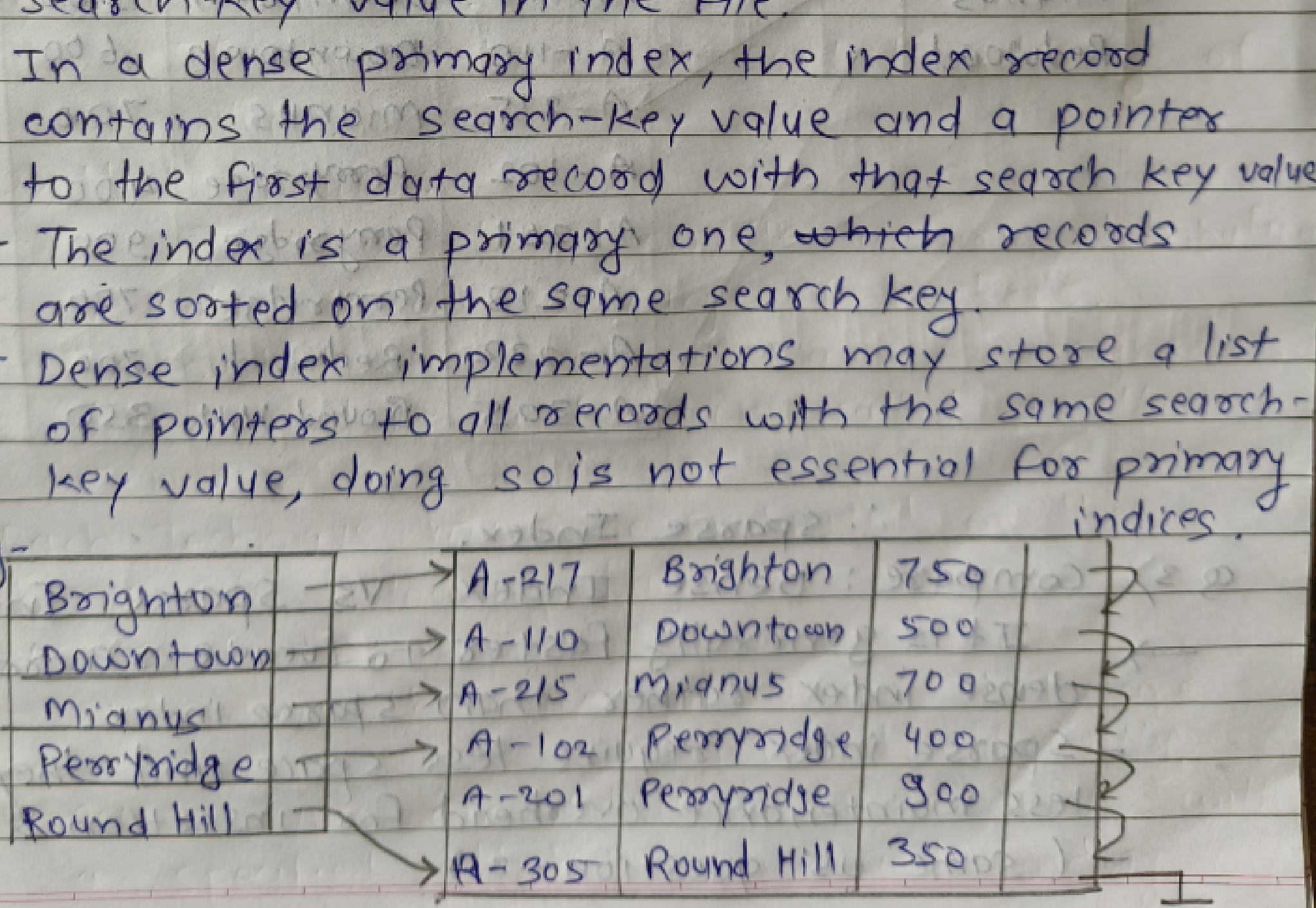
∴ These are different factors to evaluate the best index technique.

Q 2) Explain different types of Primary Indices with examples.

→ The index which produce same sequence of records as the sequence of records produced by index on primary key is called Primary Indices.
- It is one of the oldest indexing scheme used in database system.
- It is useful for applications that require both sequential processing of the file and random access to individual records.
- The primary index can be classified into 2 types
  a) Dense Index
  b) Sparse Index.

a) Dense Index :-
It is an index record appearing for every search-key value in the file.
- In a dense primary index, the index record contains the search-key value and a pointer to the first data record with that search key value
- The index is a primary one, ~~which~~ records are sorted on the same search key.
- Dense index implementations may store a list of pointers to all records with the same search-key value, doing so is not essential for primary indices.

eg-



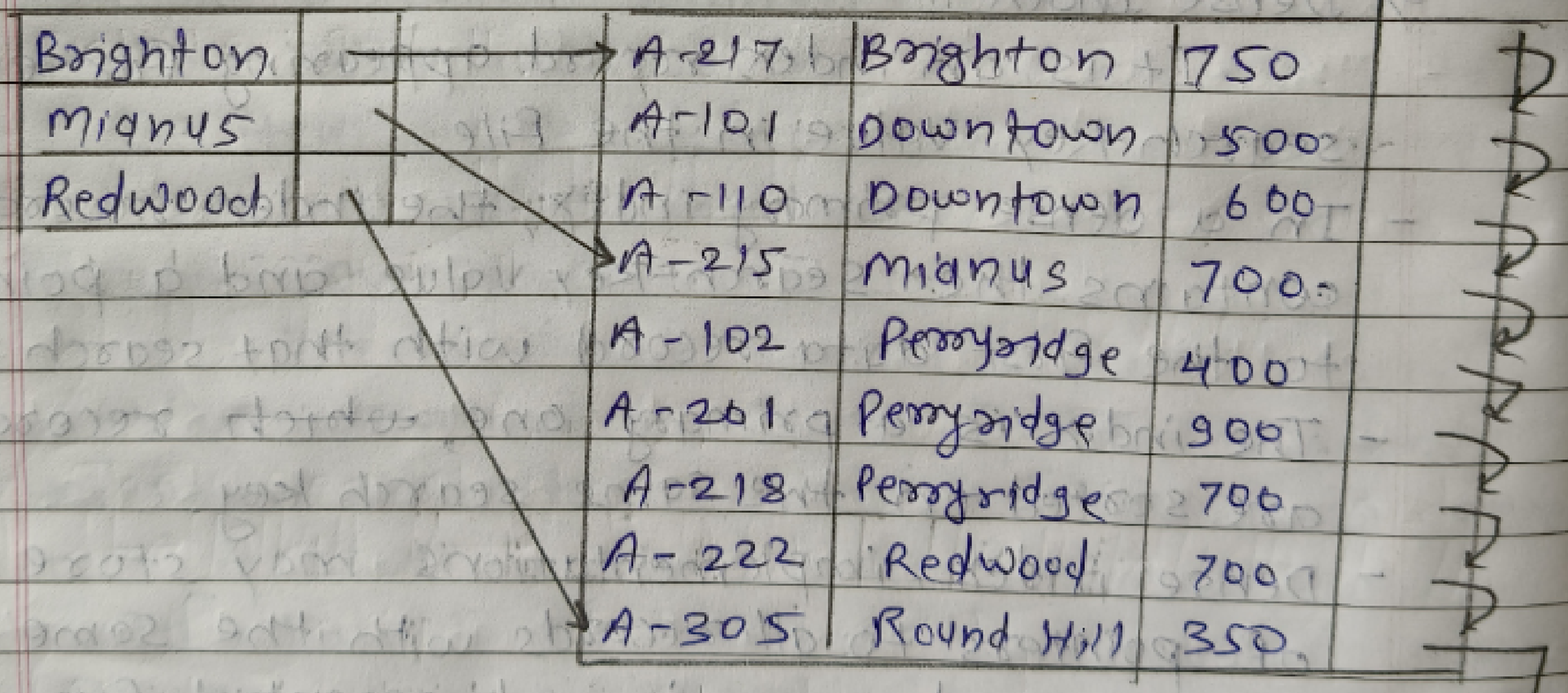

∴ Dense index.

b) Sparse Index:

It is an index records that appears for only some of the search key values.

- Each index record contains a search key value and a pointer to the first data record with that search key value.
- To locate a record, find the index entry with the largest search key value that is less than or equal to the search key value for which we are looking.
- Basically, start at the record pointed to by that index entry, and follow the pointers in the file until we find the desired record.

eg.




∴ Sparse Index.

q 3) Compare: Dense Index Vs Sparse Index.

→ It is generally faster to locate record in dense index rather than sparse index.

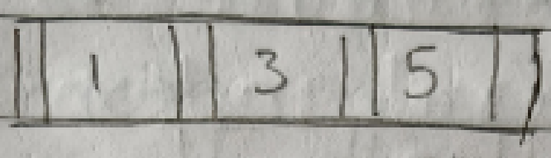
- Sparse index require less space and they impose less maintenance overhead for insertion & deletion
- (same as question - 2)

Q 4) Construct B+ Tree : 1,3,5,7,9,2,4,6,16,8,19,10.
Pointers - 4 ; keys - 3
a) Insert : 11,12,13,15
b) delete : 9,7,11.

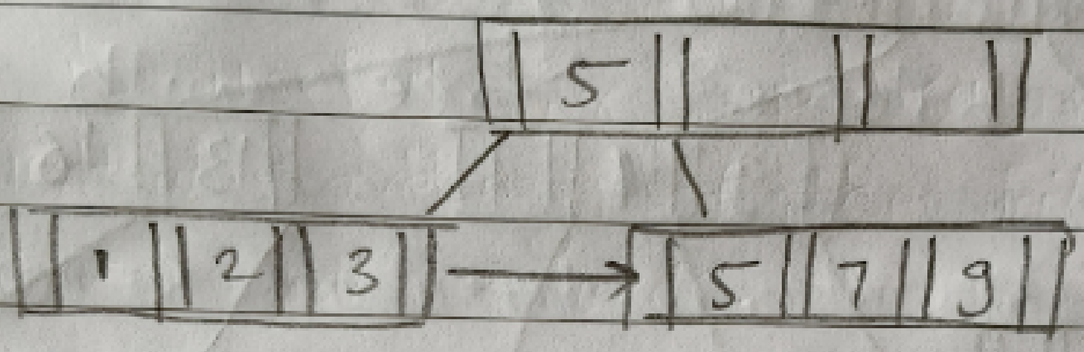
→

B+ Tree :

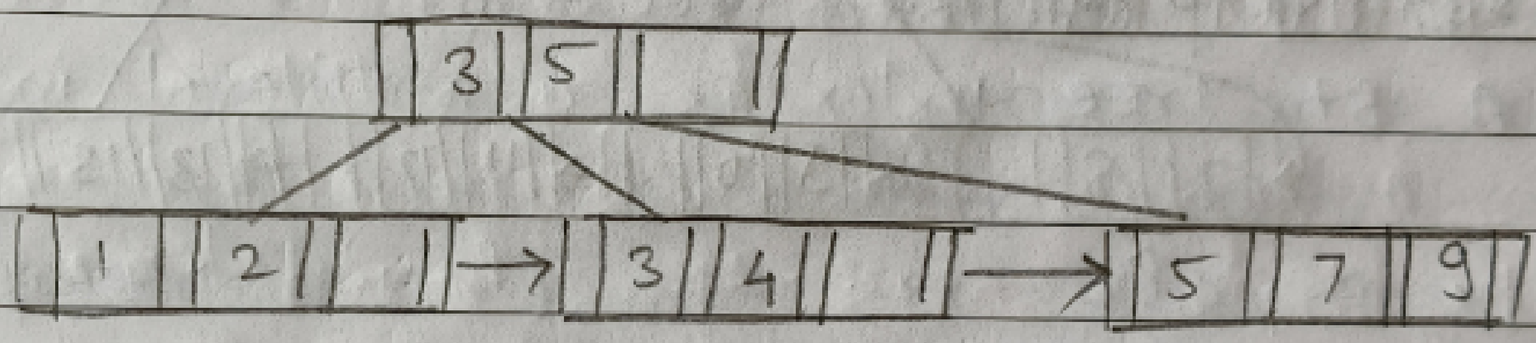
1)



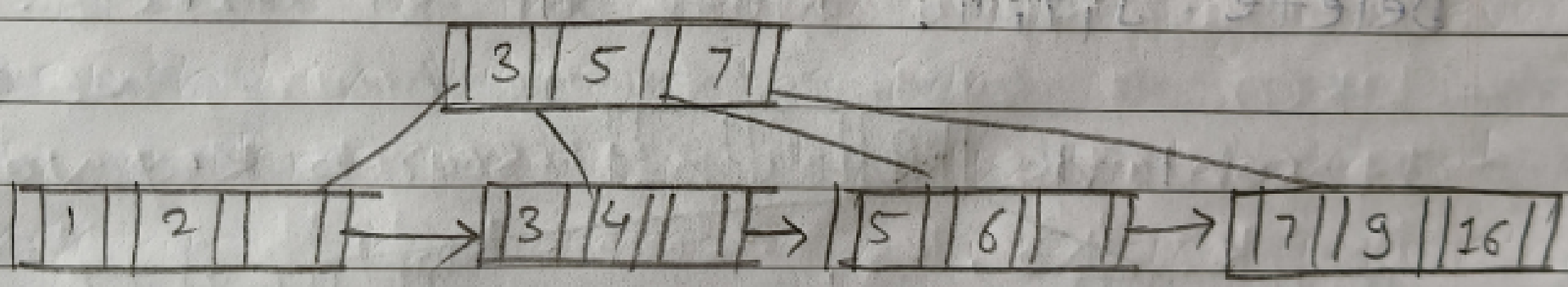

2)



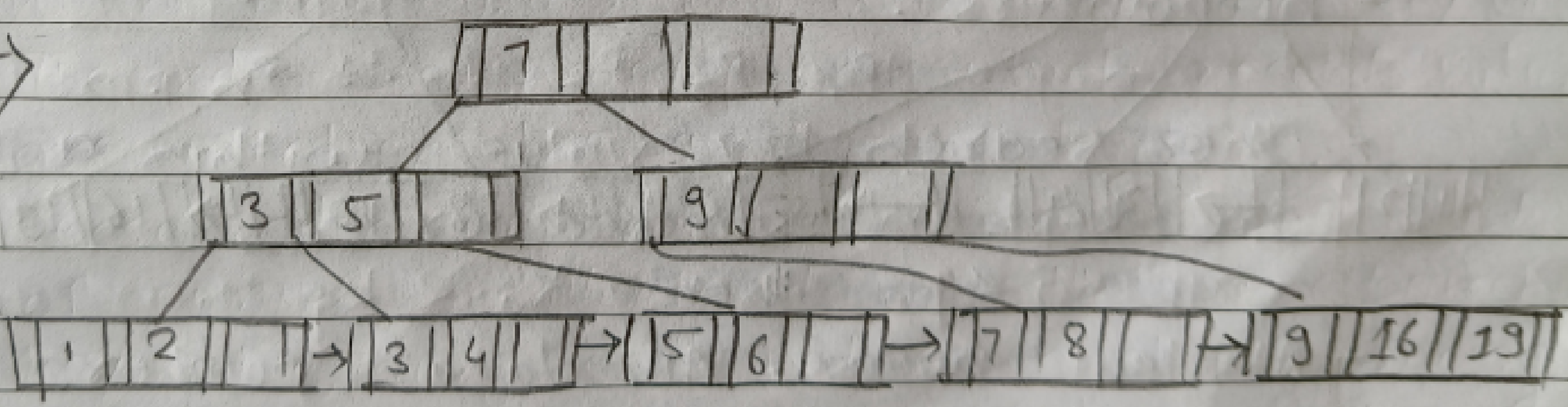

3)



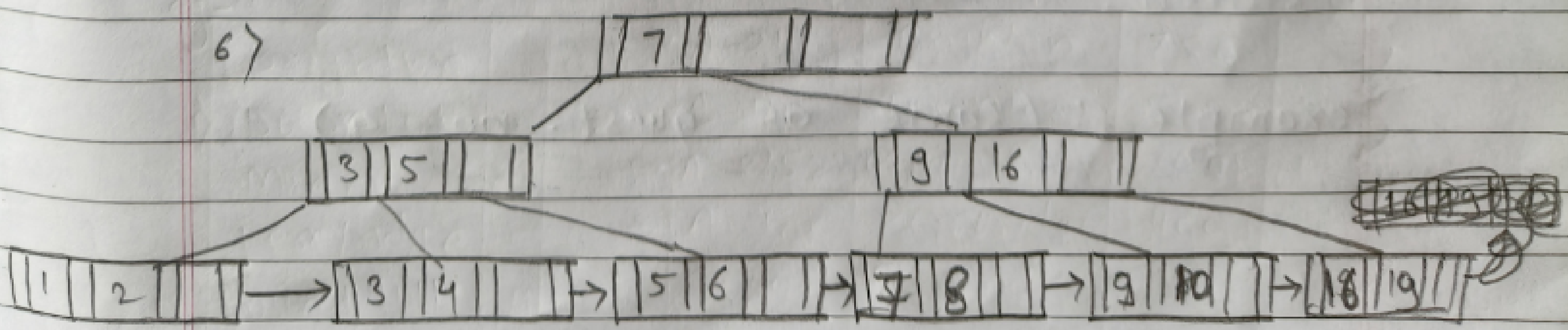

4)




5)




6)

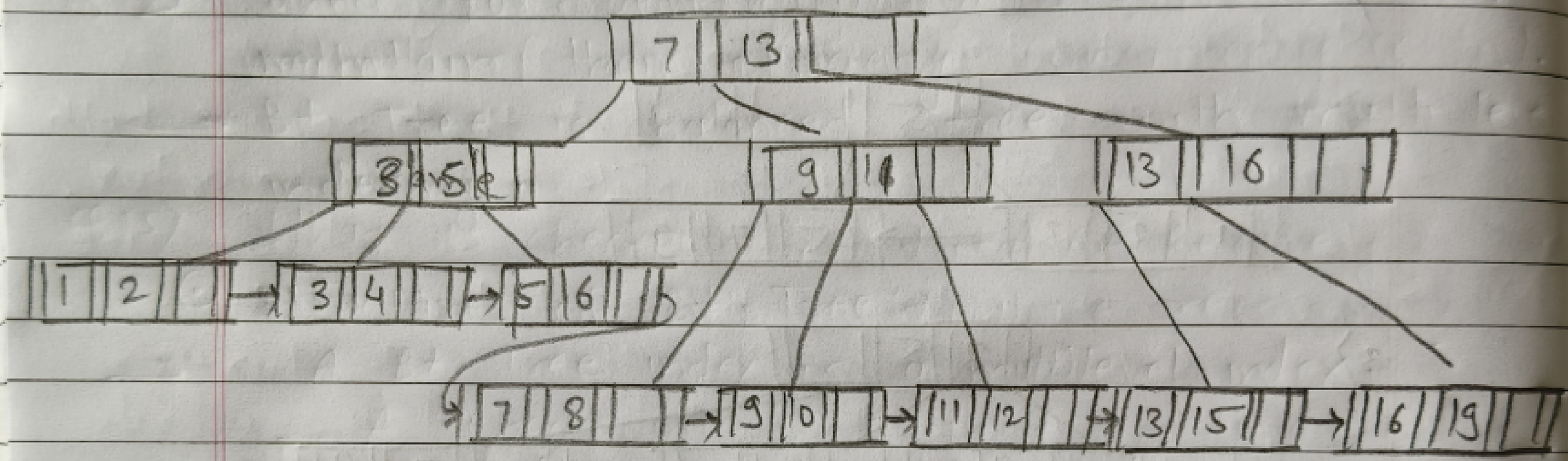



B+ Tree,

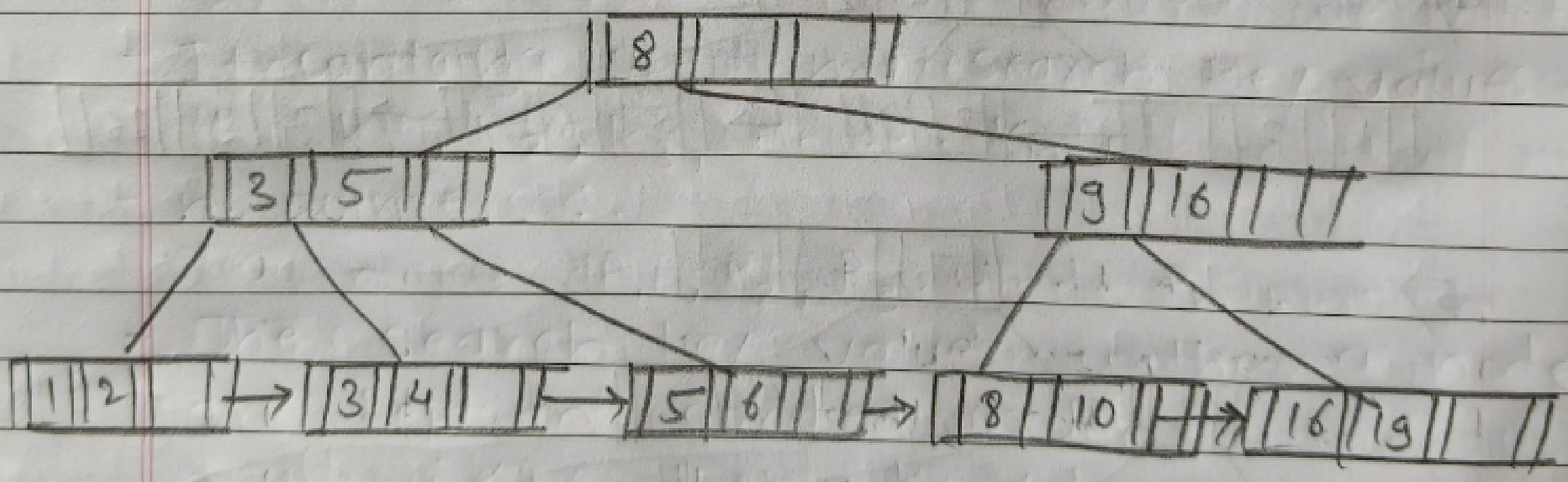
Insert: 11, 12, 13, 15.

1) Insert 11 : [9, 10, 11] : no split
2) Insert 12 : [9, 10, 11, 12] : split
3) — 11 — 13 & 15

[9, 10] [11, 12]




Delete: 9, 7, 11.

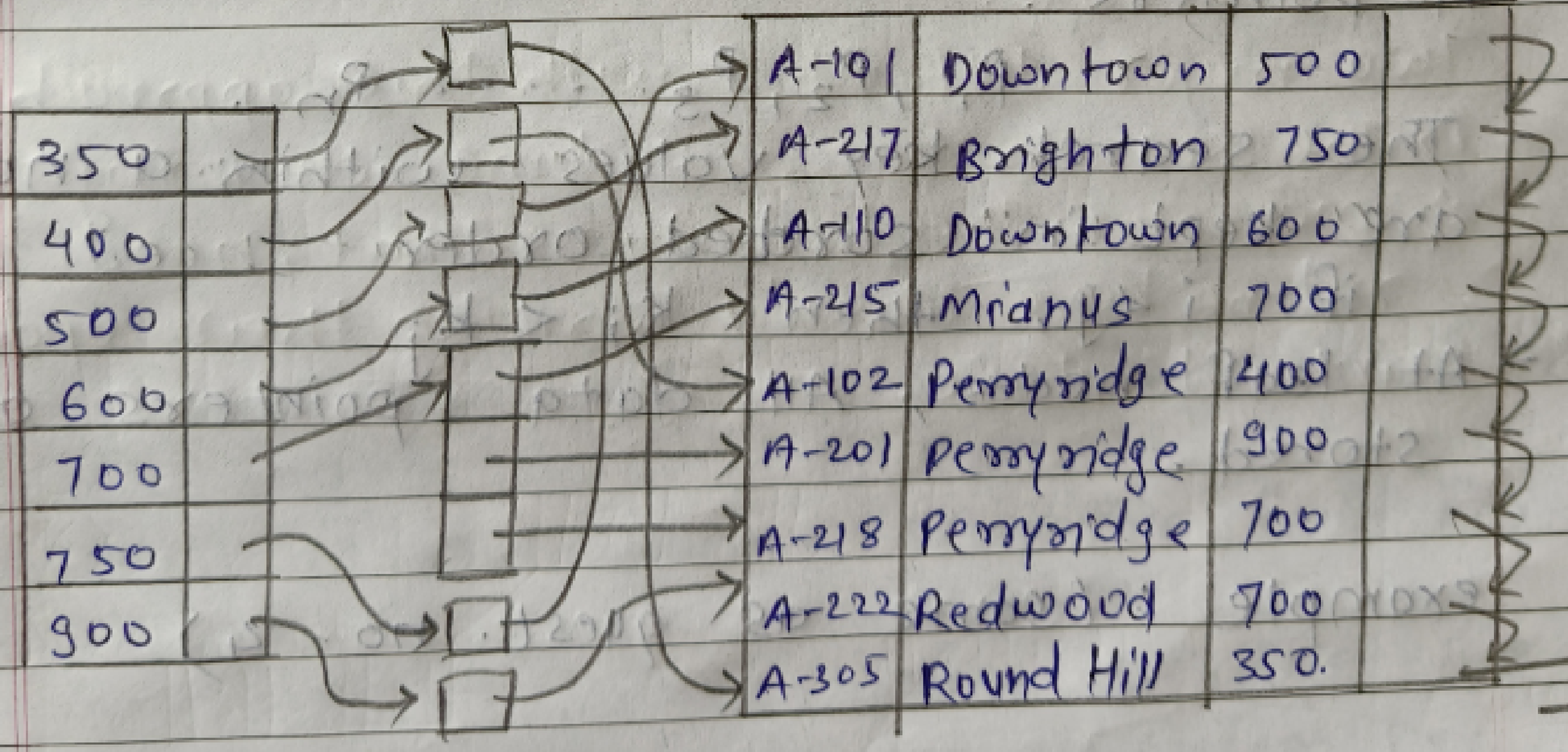



(Draw each step)
((rosscheck the ans)

Q 8) Explain secondary Indices?

→

- The Indices whose search key specifies an order different from the sequential order of the file are called secondary indices
- In secondary indices the order of rows does not match the physical order of the actual data.
- It is a non-clustered index.
- Secondary index must be dense, with an index entry for every search key value and a pointer to every record in a file.
- If a secondary index stores only some of the search key values then records with intermediate search key values may be anywhere in the file.
- Following diagram shows the secondary index on account file, on nonrandidate key balance.



Q 9) Explain B+ Tree with example.

→

A B+ Tree is a balanced multi-level tree indexing structure used in database to store and search records efficiently.

- The B+ tree structure is most widely used for several index structures that maintain their efficiency.
- B+ Tree is balanced tree with multi-level indexing.

① Structure of B+ Tree:-

A B+ Tree index is a multilevel index:

| $P_1$ | $K_1$ | $P_2$ | ..... | $P_{n-1}$ | $K_{n-1}$ | $P_n$ |
|---|---|---|---|---|---|---|

Typical Node of B+ Tree

- It contains upto n-1 search key values $K_1, K_2, K_3, \ldots K_{n-1}$.
- n pointers: $P_1, P_2, P_3 \ldots P_n$.
- The search key values within a node are kept in sorted order if $i < j$, then $K_i < K_j$
- At leaf nodes, All data pointers are stored.

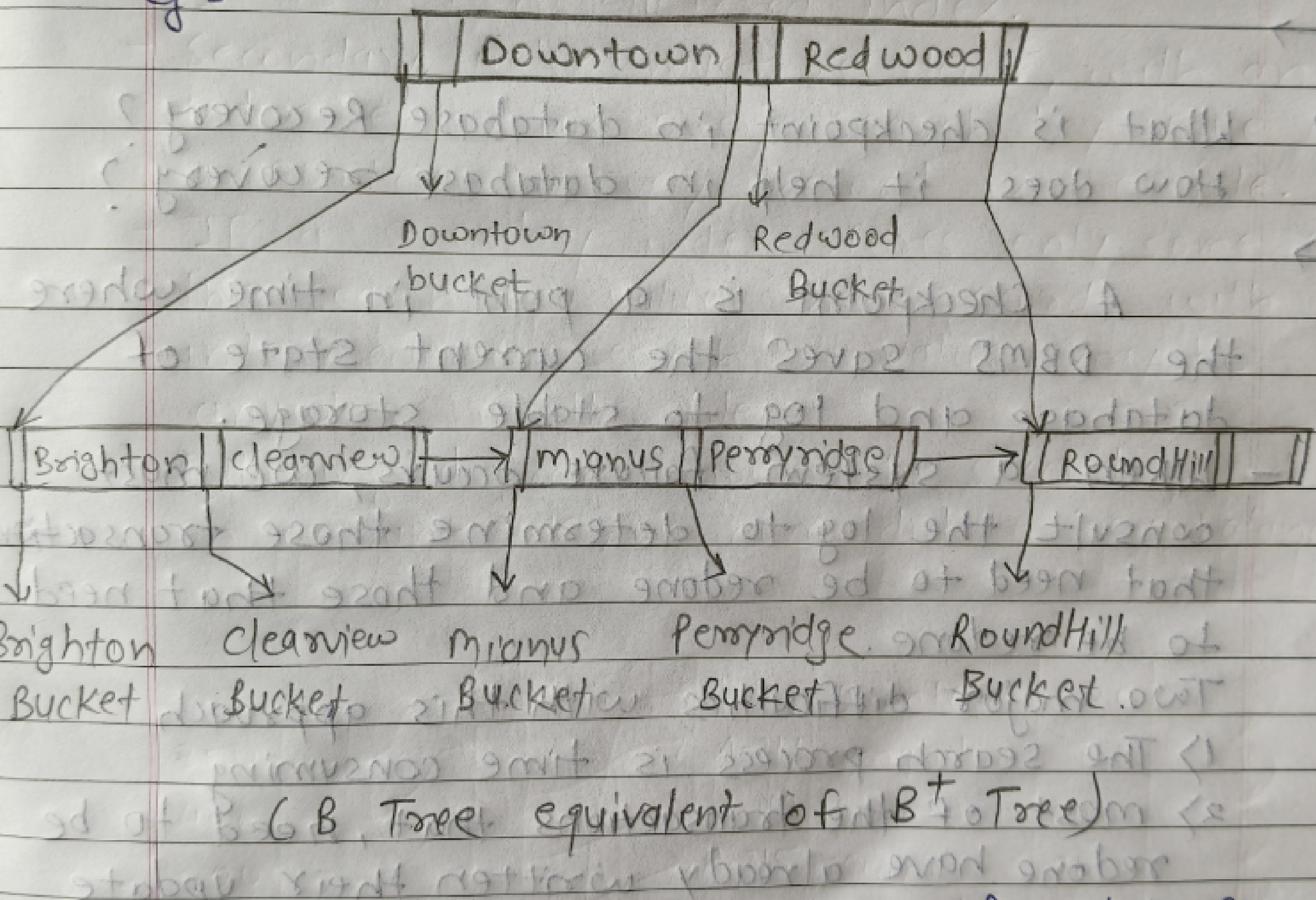
example: (same as quest. no - 4)

Q10) Explain B Tree with example:

→

A B-tree is a balanced multilevel tree data structure used for indexing in databases
- B-Tree eliminates the redundant storage of search key values.
- A B Tree allows search-key values to appear only once.

eg -




(B Tree equivalent of $B^+$ Tree)

- The keys that appear in nonleaf nodes appear nowwhere else in B-tree.
- There is an additional pointer field for each search key in a nonleaf node.
- The additional pointers point to either file records or buckets for the association key.
- In a B-Tree, the deleted entry may appear in a nonleaf node.

Properties
- Multi-level index tree
- Balanced
- key appears only once
- used when data → sorted.

8.18) What is concurrency control? methods of

DATE

PAGE NO

Q12] What is concurrency control? Name various methods of controlling the concurrency control?

→ Concurrency control is a DBMS mechanism that ensures correct & consistent execution of concurrent transactions so that database integrity is preserved.

* Methods of Concurrency Control:-

(1) Lock-Based Protocols :- Uses shared/exclusive locks to control access to data.
(2) Two-Phase Locking (2PL) :- Growing phase & shrinking phase; ensures serializability.
(3) Timestamp Ordering :- Assigns timestamp to transactions to order execution.
(4) Validation / Optimistic Concurrency control :- Checks for conflicts before commit.
(5) Serial Execution :- Transaction run one after another.

Q.13

Transactions :-

i) Transaction is a collection of operations that form a single logical unit of work. A transaction is a unit of program execution that can enable to access or update the data inside the database.

ii) It means suppose We have database of Account (bank account) then We have to transfer rs.50 from account A to B is a one kind of transaction.

Properties of transactions:-

1. Atomicity:- It says that either all operations of the transaction are reflected properly inside database or none are.
Suppose, We have two accounts as A & B having rs 1000 & rs 2000. Therefore We have transaction as transfer rs 50. from A TO B.
i.e A → A - 50 . And B → B + 50.
but suppose Failure occurs after step 1. then value of acc. A is 950 & value of acc. B is 2000. ∴ sum is 2950 i.e 50 is missing. In that case, as both transactions are not complete system will roll back to previous values.
∴ Value of acc. A = 1000 & value of acc. B = 2000.

2. Consistency:- Execution of a transaction in isolation i.e no other transaction executing conc

rrently, preserves the consistency of the database.
suppose, We ~~have~~ are transfering rs 50 from acc. A to B having initial balances 1000 & 2000.
∴ A → A-50 , B → B+50.
Therefore, the sum before transaction is 1000 + 2000 = 3000. And sum after transaction is 950 + 2050 = 3000. But if the interrupt or any error is occured then it may be inconsistent.

3. Isolation :- It states that even though multiple transactions may occur concurrently. the system guarantees that for every pair i.e suppose Ti & Tj are transactions then Tj will start its execution either before or after the complete execution of Ti.
suppose We have transaction t1 as transfer rs 50 from A to B having initial balances as 1000 & 2000. And transaction t2 is read the values of both accounts.

∴ If transaction t1 occurs as:-
A → A-50 And B → B+50
= 1000-50 = 2000+50
= 950 = 2050

then suppose t2 will executes in middle of t1 transaction then it will get wrong result as A → 950, B → 2000.

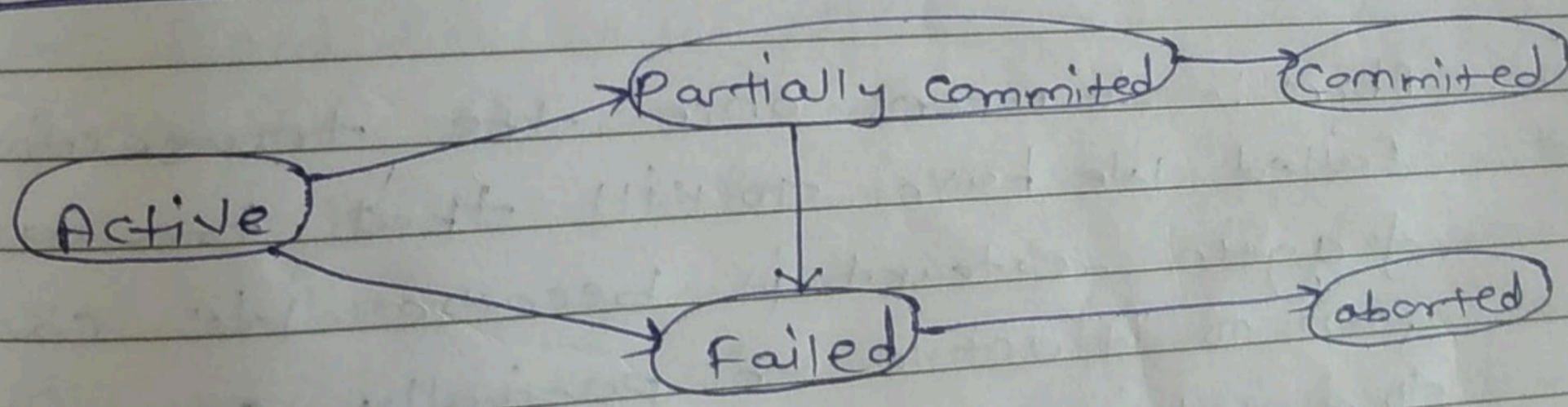
but if Suppose t2 will be executed before t1 then it will get correct result as 1000. & 2000. And if after t1 then also it will get correct result as 900 & 2100.

∴ read(X):- Transfers the data item X from the database to a local buffer variable.

Write(X):- Transfers the data item x from local buffer Variable to the database.

4. Durability:- This property says that it will guareentees that once a transaction completes successfully, All the updates that are carried on the database will stay as it is even if there is System Failure occurs.

- Transaction States:- Q.14)




Active:- Whenever We're performing transactions then that transaction is in active state. i.e CPU is executing that transaction with the help of RAM.

eg:- Read(A) ; Write(A).

Partially commit :- It means we're performing all read & write operations but these are not saved permanently until we execute commit commited then partially commit means we have done all operations but we're not performing Commit Command.

Commited :- It means whatever the changes or operation we had done then it should be done inside the hard disk or we're performing commit Command & because of that all changes has been saved in database.

Failed :- It means because of some problems like hardware or anything else transaction fails. It should be failed after active state or after the partially commited state.

Abort :- It means once the transaction is failed we have to kill that transaction & again restart it because we can't directly pass to it active or partially commited state to maintain the consistency we've to restart transaction.

- Failure classification :-
  There are different types of failures because of which system gets fail.

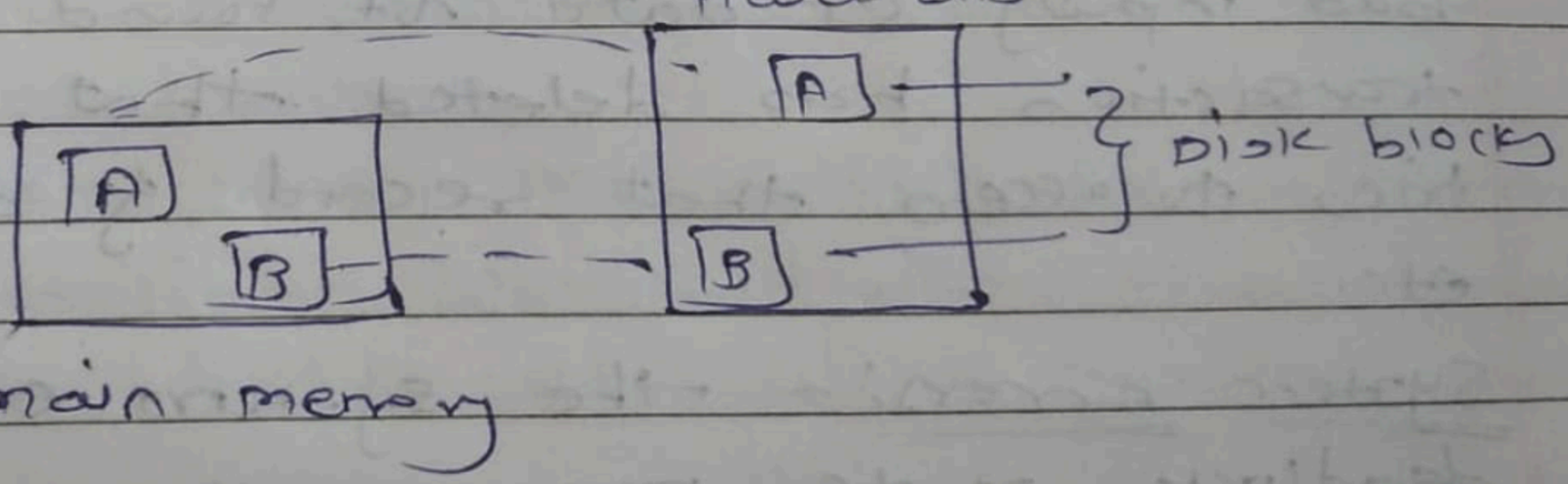
i) Transaction Failure:- due to some reasons the transaction gets fail. that reason is like logical error or system error.
logical error:- these error are related to some internal conditions or We're giving bad input or data not found means are transaction has deleted that record other tries to access that record & data overFlow etc.

System error:- the system entered in the deadlock state means there are 2 transaction T1 & T2. And T1 requests T2 for other resources & T2 requests T1 for some resources because of that System goes in deadlock (both T1 & T2 waits for long time).

ii) system crash:- There is hardWare malfunctions means there should be problem in hardware, or hard disk will get crash, And Bug in database software or operating System it means. because of some coding problem there should be bug or error in the database s/w or because of some bug or Virus is entered in the operating system. Then because of all that System related problem is created.

iii) Disk Failure:- Here, Suppose, in the Hard disk there are two disk blocks A & B & We have to perform transaction on that two then We're taking that in the main memory & performing transaction. once transaction is performed then this

We perform commit operation We're storing that block values again in the Hard disk but due to some reason. there should be possibility that disk blocks (Hard disk blocks) loses its content. because of which failure occurs.



Q15) What is checkpoint in database Recovery? How does it help in database recovery?

→

A Checkpoint is a point in time where the DBMS saves the current state of database and log to stable storage.

- When a system failure occurs, we must consult the log to determine those transactions that need to be redone and those that need to be redone.

Two major difficulties with this approach

1) The search process is time consuming
2) most of the transactions that, need to be redone have already written their update into database.

- Why checkpoint is Needed ?

→ When a system failure occurs, the DBMS must check the log to decide which transactions to redo or undo. Without checkpoints, the system would need to search the entire log, which is time-consuming.

- How does it help ?

→ Checkpoint makes crash recovery faster & easier by reducing the amount of log to be scanned.

- At checkpoint, transactions are not allowed to perform update actions.
- A <checkpoint> record is written in the log.
- DBMS scans backward only up to the last checkpoint to identify active transaction
- Only those transactions that started before or after that checkpoint & have not committed are undone or redone.

* Example :-

- If checkpoint occurred during T67, only T67 to T100 need to be considered.
- Committed ones are redone.
- Uncommited ones are Undone.

This avoids reprocessing all past transactio improving recovery time.

## Deferred Database Modification:

The **deferred-modification technique** ensures transaction atomicity by recording all database modifications in the log,

Q.16)

deferring the execution of all write operations of a transaction until the transaction partially commits.
The execution of transaction $T_i$ proceeds as follows
Before $T_i$ starts its execution, a record $<T_i$ start$>$ is written to the log.
A write($X$) operation by $T_i$ results in the writing of a new record to the log.
Finally, when $T_i$ partially commits, a record $<Ti$ commit$>$ is written to the log.

$T_0$ be a transaction that transfers \$50 from account $A$ to account $B$

$T_0$: read($A$);
$A := A - 50$;
write($A$);
read($B$);
$B := B + 50$;
write($B$).

Let $T_1$ be a transaction that withdraws \$100 from account $C$:

$T_1$: read($C$);
$C := C - 100$;
write($C$).

<$T_0$ start>
<$T_0$, $A$, 950>
<$T_0$, $B$, 2050>
<$T_0$ commit>
<$T_1$ start>
<$T_1$, $C$, 600>
<$T_1$ commit>

Portion of the database log corresponding to $T_0$ and $T_1$

the valu e only after the record <$T_0$, $A$, 950> has been placed in the log.

| Log | Database |
|---|---|
| <$T_0$ start> | |
| <$T_0$, $A$, 950> | |
| <$T_0$, $B$, 2050> | |
| <$T_0$ commit> | |
| | $A = 950$ |
| | $B = 2050$ |
| <$T_1$ start> | |
| <$T_1$, $C$, 600> | |
| <$T_1$ commit> | |
| | $C = 600$ |

State of the log and database corresponding to $T_0$ and $T_1$.

Using the log, the system can handle any failure that results in the loss of information on volatile storage. redo($T_i$) sets the value of all data items updated by transaction $Ti$ to the new values.

The set of data items updated by $T_i$ and their respective new values can be found in the log.

After a failure, the recovery subsystem consults the log to determine which transactions need to be redone.

## Q.17) Immediate Database Modification

The **immediate-modification technique** allows database modifications to be output to the database while the transaction is still in the active state.

Data modifications written by active transactions are called **uncommitted modifications**.

Before a transaction *Ti* starts its execution, the system writes the record <*Ti* start> to the log.

<$T_0$ start>
<$T_0$, *A*, 1000, 950>
<$T_0$, *B*, 2000, 2050>
<$T_0$ commit>
<$T_1$ start>
<$T_1$, *C*, 700, 600>
<$T_1$ commit>

Portion of the system log corresponding to $T_0$ and $T_1$.

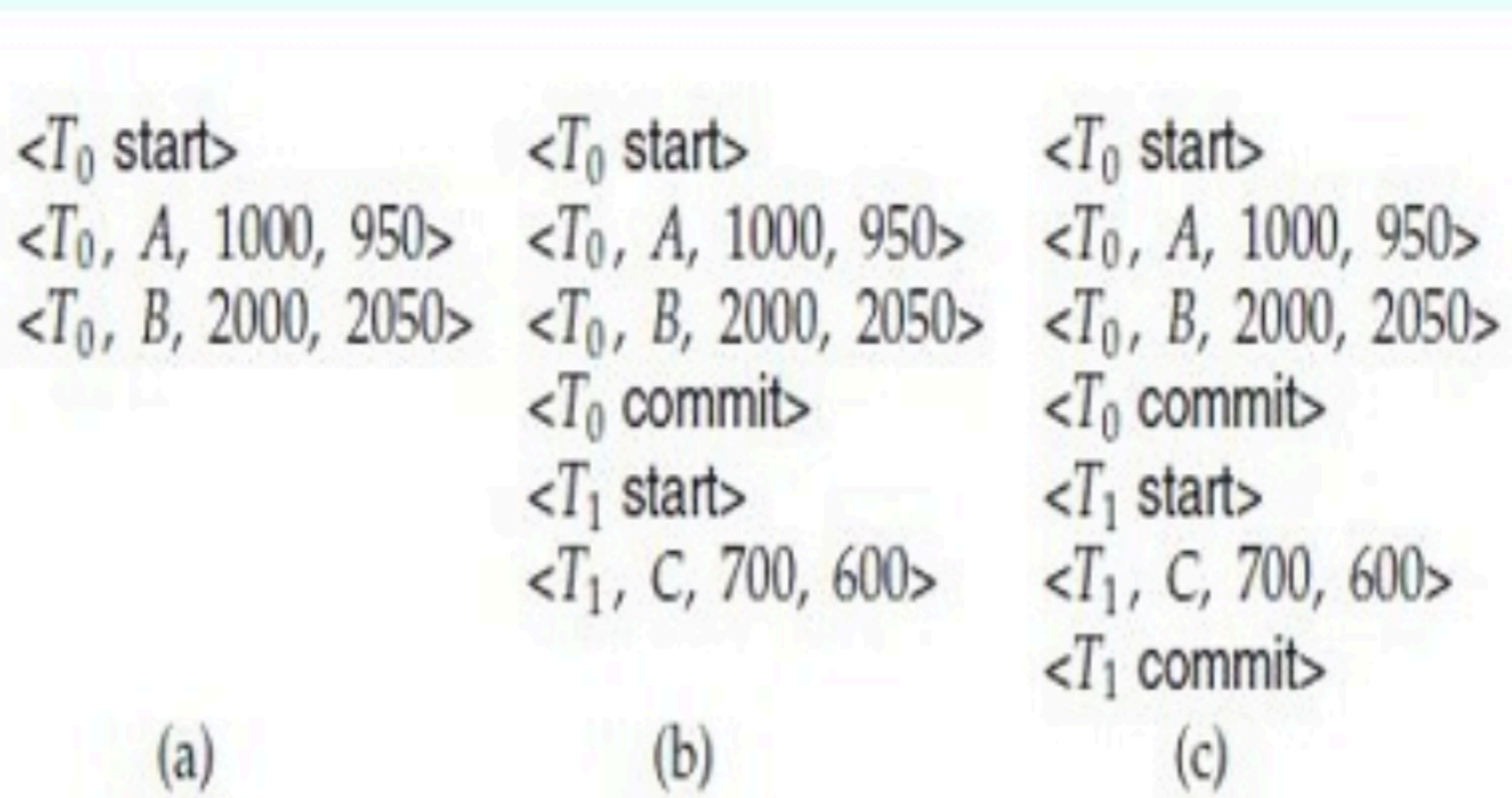
| Log | Database |
|---|---|
| $<T_0$ start$>$ | |
| $<T_0, A, 1000, 950>$ | |
| $<T_0, B, 2000, 2050>$ | |
| | $A = 950$ |
| | $B = 2050$ |
| $<T_0$ commit$>$ | |
| $<T_1$ start$>$ | |
| $<T_1, C, 700, 600>$ | |
| | $C = 600$ |
| $<T_1$ commit$>$ | |

State of system log and database corresponding to $T_0$ and $T_1$.

The system can handle any failure that does not result in the loss of information in nonvolatile storage. The recovery scheme uses two recovery procedures:

- undo($T_i$) restores the value of all data items updated by transaction $T_i$ to the old values.
- redo($T_i$) sets the value of all data items updated by transaction $T_i$ to the new values.

- Transaction $T_i$ needs to be undone if the log contains the record $<T_i$ start$>$, but does not contain the record $<T_i$ commit$>$.
- Transaction $T_i$ needs to be redone if the log contains both the record $<T_i$ start$>$ and the record $<T_i$ commit$>$.




The same log, shown at three different times.

## Q.20) Log-Based Recovery

The most widely used structure for recording database modifications is the **log**.

The log is a sequence of **log records,** recording all the update activities in the database.

An **update log record** describes a single database write.

- **Transaction identifier** is the unique identifier of the transaction that performed the write operation.
- **Data-item identifier** is the unique identifier of the data item written. Typically, it is the location on disk of the data item.
- **Old value** is the value of the data item prior to the write.
- **New value** is the value that the data item will have after the write.

# various types of log records

- $\langle T_i$ start$\rangle$. Transaction $T_i$ has started.
- $\langle T_i, X_j, V_1, V_2\rangle$. Transaction $T_i$ has performed a write on data item $X_j$. $X_j$ had value $V_1$ before the write, and will have value $V_2$ after the write.
- $\langle T_i$ commit$\rangle$. Transaction $T_i$ has committed.
- $\langle T_i$ abort$\rangle$. Transaction $T_i$ has aborted.

log records to be useful for recovery from system and disk failures,
the log must reside in stable storage.
two techniques for using the log to ensure transaction atomicity despite failures.

**Deferred Database Modification:**

The **deferred-modification technique** ensures transaction atomicity by recording all database modifications in the log,